

Массивы и методы `map`, `filter`, `find`

Пошаговая отработка практических задач: от простой проекции примитивов до многоуровневой фильтрации каталога, поиска по предикатам и построения пайплайнов.

`> zsh — node array-lab.js`

```
# Запускаем лабораторный скрипт для отработки методов  
массивов
```

```
→ node array-lab.js
```

```
✓ Метод .map() протестирован: 100% элементов трансформированы без  
мутации
```

```
✓ Метод .filter() протестирован: предикаты отработали  
корректно
```

```
✓ Метод .find() протестирован: ранний выход зафиксирован
```

01

Подготовка тестового стенда

Создание файла для практики и подключение базового набора тестовых данных

Создадим тестовый скрипт с реалистичным массивом товаров интернет-магазина, на котором будем отрабатывать все последующие задачи.

+ 1.1. Инициализация файла `array-lab.js`

```
// Базовый массив товаров (эмуляция ответа сервера):  
const products = [ { id:  
  "p1", title:  
  "Механическая клавиатура", price:  
  8500, category:  
  "accessories", inStock:  
  true, rating:
```

```
4.8 }, { id:
  "p2", title:
  "Игровой монитор 144Hz", price:
  24000, category:
  "displays", inStock:
  false, rating:
4.6 }, { id:
  "p3", title:
  "Беспроводная мышь", price:
  4200, category:
  "accessories", inStock:
  true, rating:
4.9 }, { id:
  "p4", title:
  "USB-C Хаб 7-in-1", price:
  3100, category:
  "adapters", inStock:
  true, rating:
4.2 }, { id:
  "p5", title:
  "Кронштейн для монитора", price:
  5600, category:
  "accessories", inStock:
  false, rating:
4.7 } ]; console.log(`Загружено товаров: ${products.length}`);
```

Обработка метода .map(): Трансформация структуры

Проекция данных: извлечение полей, пересчет валют и генерация UI-объектов

Метод `.map()` преобразует каждый элемент массива в новый вид, не мутируя оригинал ($N \rightarrow N$).

</> 2.1. Задачи на проекцию данных

JS mapTasks.js

```
// Задача 1: Получить простой массив только из названий всех
// товаров:
const productTitles = products.map(item => item.title); console.log("1. Названия товаров:", productTitles);

// Задача 2: Добавить скидку 15% и отформатировать цену для
// отображения:
const productsWithDiscount =
products.map(item => ({ ...item,
discountPrice: item.price * 0.85,
formattedPrice:
```

```
formattedPrice:
  `${item.price.toLocaleString("ru-RU")} ₽`
}));

// Задача 3: Превратить массив объектов в массив JSX-строк
(эмуляция React List):
const mockJsxList = products.map((item, index) =>
  `<li key="${item.id}" data-index="${index}">${item.title} –
    ${item.price}₽</li>`
); console.log("3. Эмуляция JSX:", mockJsxList[0]);
```

i Важное правило React

Внутри метода `.map()` при возврате JSX всегда указывайте проп `key={item.id}` на самом внешнем теге, чтобы React мог точно обновлять элементы списка при изменениях.

Обработка метода `.filter()`: Выборки и удаление

Создание условий по наличию, ценовым диапазонам и удаление по ID

Метод `.filter()` возвращает новый массив, оставляя только элементы, для которых колбэк вернул `true`.

3.1. Практические сценарии фильтрации

filterTasks.js

```
// Задача 1: Выбрать товары, которые есть в наличии (inStock === true):
const inStockOnly = products.filter(item => item.inStock); console.log(`1. В наличии товаров: ${inStockOnly.length}`);

// Задача 2: Фильтр по категории и бюджету (accessories дешевле 60000₽):
const affordableAccessories =
products.filter(item => item.category === "accessories" && item.price < 60000 ); console.log("2. Бюджетные аксессуары:", affordableAccessories);
```

```
// Задача 3: Удаление товара по ID (основа handleDelete в React
State):
const idToDelete =
  "p3";
const remainingProducts =
  products.filter(item => item.id !==
  idToDelete); console.log("3. Товары после удаления p3:", remainingProducts.length); // 4 товара
```

04

Обработка метода `.find()`: Поиск конкретного элемента

Быстрое извлечение объекта по ID и обработка отсутствующих значений

Метод `.find()` находит первый совпавший элемент и сразу останавливает обход массива. Если совпадений нет — возвращает `undefined`.

🔍 4.1. Поиск по уникальному ID и предикатам

```
JS findTasks.js

// Задача 1: Найти товар по точному ID (для страницы отдельного
// товара):
const targetId =
  "p4";
const foundProduct = products.find(item => item.id === targetId);

// Безопасное чтение с оператором ?. и ??:
console.log(`Найден: ${foundProduct?.title ?? "Товар не найден"}`);
```

```
// Задача 2: Поиск первого товара с высоким рейтингом ( $\geq$ 
  4.9):
const topRated = products.find(item  $\Rightarrow$  item.rating  $\geq$  4.9);
console.log("Топ товар:", topRated?.title);
// "Беспроводная мышь"

// Задача 3: Поиск несуществующего элемента:
const missing = products.find(item  $\Rightarrow$  item.id  $\equiv$  "p999");
console.log("Результат поиска несуществующего:", missing); // undefined
```

Связывание методов позволяет решить комплексную задачу фильтрации и форматирования за один понятный проход без промежуточных мутаций.

🔗 5.1. Реализация витрины «Хиты продаж»

```
chainingPipeline.js

// Задача: Отобрать товары В НАЛИЧИИ с рейтингом ≥ 4.5,
// отсортировать и вернуть красивый формат карточки:
const showcaseProducts = products
  .filter(item => item.inStock &&
    item.rating ≥ 4.5) .map(item => ({ badge: "★ Хит",
    displayName:
      `${item.title} (${item.rating})`,
    finalPrice:
      `${item.price.toLocaleString("ru-RU")} ₽`
  })); console.log("Витрина хитов:", showcaseProducts);
```

Нажимайте кнопки для запуска разных методов над массивом `products` :

↶ Выполнить `.map()`

▼ Выполнить `.filter(inStock)`

🔍 Выполнить `.find(id: p2)`

🔗 Запустить цепочку

```
// Нажмите любую кнопку выше для выполнения JS-метода над  
коллекцией ...
```

Итоговое практическое задание: CatalogManager

Самостоятельное создание модуля управления каталогом для React

Объединим методы `map`, `filter` и `find` в готовый модуль бизнес-логики:

```
catalogManager.js — Итоговый скрипт

// 1. Поиск товара по ID:
const
getProductById = (list, id) =>
list.find(item => item.id === id);

// 2. Фильтрация по категории с опциональным фильтром
наличия:
const
filterCatalog = (list, { category,
onlyInStock = false }) => {
return list.filter(item => { const matchesCategory
= category ? item.category === category :
true;
```

```
    item.inStock : true;  
    return matchesCategory &&  
    matchesStock; }); };
```

```
// 3. Быстрое изменение цены товара по ID через .map():
```

```
const  
updateProductPrice = (list, id, newPrice)  
⇒ { return list.map(item ⇒ item.id === id ? { ...item, price: newPrice } : item);  
};
```

```
// Проверка работы модуля:
```

```
const activeAccessories =  
filterCatalog(products, { category:  
"accessories", onlyInStock:  
true });  
const updatedList =  
updateProductPrice(products,  
"p1", 7990);  
console.log("Аксессуары в наличии:", activeAccessories.length);  
// 2 товара console.log("Новая цена p1:",  
getProductById(updatedList,  
"p1")?.price);  
// 7990
```